QuantumCircuit.compose errors when qubits argument is numpy.int64 ins

https://github.com/Qiskit/qiskit/issues/8691

ROW 93

`QuantumCircuit.compose` errors when `qubits` argument is `numpy.int64` instead of `int`. #8691

New issue

Closed #9206

tomohiro-soejima opened on Sep 6, 2022 · edited by tomohiro-soejima Edits ···

Environment

- `qiskit-terra: '0.21.0':
- `python: '3.9.12'`:
- Ubuntu-20.04 windows subsystem for Linux running on Windows 10:
- `numpy: 1.21.5`

What is happening?

`QuantumCircuit.compose()` errors when `qubits` argument is a list of `numpy.int64` instead of `int`.

How can we reproduce the issue?

```
from qiskit import QuantumCircuit
from qiskit.converters.circuit_to_dag import circuit_to_dag
import numpy as np

qc0 = QuantumCircuit(1)
qc1 = QuantumCircuit(1)
qc0.x(0)
qc1.x(0)

# First, let's try qubits = [0] (vanilla python `int`)
qc_vanilla_int = qc0.compose(qc1, qubits=[0])
# `QuantumCircuit.compose` does not error even when the `qubits` argument is invalid, so we need to check the
circuit_to_dag(qc_vanilla_int) # does not error

# Now let's try qubits = [np.int64(0)]
qc_numpy_int = qc0.compose(qc1, qubits=[np.int64(0)]) #np.arange(0) does the same thing.
circuit_to_dag(qc_numpy_int) #this one errors
```

Here is the error:

```
DAGCircuitError: "(qu)bit 0 not found in OrderedDict([(Qubit(QuantumRegister(1, 'q'), 0),
DAGOutNode(wire=Qubit(QuantumRegister(1, 'q'), 0)))])"
```

What should happen?

`qc0.compose` should return a `QuantumCircuit` object that can be converted to a DAG.

Any suggestions?

No response

tomohiro-soejima added bug on Sep 6, 2022

jakelishman on Sep 6, 2022 · edited by jakelishman Edits Member ···

Yeah, this isn't the first time `QuantumCircuit.compose` has got caught doing its own thing and ignoring the conversions / book-keeping done by other parts of the class. I just flicked through it and I can see a bunch of places where it's liable to explode as well. I'll overhaul it a little bit to fix some of these niggles.

edit: thanks for the report!

1

Assignees
jakelishman

Labels
bug

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
Fix `QuantumCircuit.compose` on unusual types and registers
Qiskit/qiskit

Notifications Customize
Subscribe
You're not receiving notifications from this thread.

Participants

jakelishman on Sep 6, 2022 · edited by jakelishman — Edits — Member

Yeah, this isn't the first time `QuantumCircuit.compose` has got caught doing its own thing and ignoring the conversions / book-keeping done by other parts of the class. I just flicked through it and I can see a bunch of places where it's liable to explode as well. I'll overhaul it a little bit to fix some of these niggles.

edit: thanks for the report!

👍 1

jakelishman self-assigned this on Sep 6, 2022

mrossinek on Sep 28, 2022 — Member

I just ran into a (related?) issue when using the new `Estimator` class. To reproduce, do the following:

```
import numpy as np
from qiskit.algorithms.minimum_eigensolvers import VQE
from qiskit.algorithms.optimizers import SLSQP
from qiskit.circuit.library import ExcitationPreserving
from qiskit.primitives import Estimator
from qiskit.quantum_info import SparsePauliOp


optimizer = SLSQP(maxiter=100)
wavefunction = ExcitationPreserving(np.int64(4))
solver = VQE(Estimator(), wavefunction, optimizer)
operator = SparsePauliOp("XXII")
solver.compute_minimum_eigenvalue(operator)
```

This results in an error arising from here: https://github.com/Qiskit/qiskit-terra/blob/4a857f61146fb185878abd8634524941d71f1f6b/qiskit/circuit/instruction.py#L76

jakelishman mentioned this on Nov 25, 2022

Fix `QuantumCircuit.compose` on unusual types and registers #9206

mergify closed this as completed in #9206 on Jan 11, 2023

Fix `QuantumCircuit.compose` on unusual types and registers #9206

Code

Merged mergify merged 8 commits into `Qiskit:main` from `jakelishman:fix-compose` on Jan 11, 2023

Conversation 9 · Commits 8 · Checks 0 · Files changed 4 · +137 −88

jakelishman commented on Nov 25, 2022 • edited — Member

Summary

`QuantumCircuit.compose` previously had its own handling for converting bit specifiers into bit instances, meaning its functionality was often surprisingly less permissive than `append`, or just incorrect. This swaps the method to use the standard conversion utilities.

The logic for handling mapping conditional registers was previously the old logic in `DAGCircuit`, which was called without respect to re-ordering of the clbits during the composition. The existing test case surrounding this that is modified by this commit made an incorrect assertion; in general, the condition was not the same after the composition. In this particular test case it was ok because both bits were being tested for the same value, but had the condition value been 0b01 or 0b10 it would have been incorrect.

This commit updates the register-mapping logic to respect the reordering of clbits, and to create a new aliasing register to ensure the condition is represented exactly, if this is required. This fixes a category of bug where too-small registers could incorrectly be mapped to larger registers. This was not a valid transformation, because it created assertions about bits that were previously not involved in the condition's comparison.

Details and comments

Fix #6583
Fix #6584
Fix #8691

Note that with this change in place, the circuit visualisation tools may well not display the circuits correctly; they don't handle aliasing registers well at the moment. Similarly, I don't know precisely how well these objects will transport through the legacy Qobj paths; the circuits being described here may well not be representable in the Qobj model. If that is the case, though, it'll be up to `assemble` to raise a suitable error; the representation here is what is correct for `QuantumCircuit`.

Fix `QuantumCircuit.compose` on unusual types and registers … — Verified — 5f96de6

jakelishman added the Changelog: Bugfix label on Nov 25, 2022

jakelishman added this to the **0.23.0** milestone on Nov 25, 2022

jakelishman requested a review from **a team** as a code owner 3 years ago

qiskit-bot commented on Nov 25, 2022 — Collaborator

Thank you for opening a new pull request.

Before your PR can be merged it will first need to pass continuous integration tests and be reviewed. Sometimes the review process can be slow, so please be patient.

While you're waiting, please feel free to review other open PRs. While only a subset of people are authorized to approve pull requests for merging, everyone is encouraged to review open pull requests. Doing reviews helps reduce the burden on the core team and helps make the project's code better for everyone.

One or more of the the following people are requested to review this:

- @Qiskit/terra-core

Reviewers
Cryoris ✓

Assignees
Cryoris

Labels
Changelog: Bugfix

Projects
None yet

Milestone
0.23.0

Development
Successfully merging this pull request may close these issues.
- `QuantumCircuit.compose` errors when `qubi...`
- Cannot convert condition in circuit with multi...
- Flattened conditional circuits give incorrect r...

Notifications — Customize
Subscribe
You're not receiving notifications from this thread.

4 participants